

Multi-Class Text Classification: A Comparative Study of Word Representations and Machine Learning and Neural Network Models

Mohammad Ifaz Alamgir

Dept of

Computer Science and Engineering
BRAC University

Dhaka, Bangladesh

mohammad.ifaz.alamgir@g.bracu.ac.bd

Umme Hani Mir Upoma

Dept of

Computer Science and Engineering
BRAC University

Dhaka, Bangladesh

umme.hani.mir.upoma@g.bracu.ac.bd

Mohammed Sayed Sameer

Dept of

Computer Science and Engineering
BRAC University

Dhaka, Bangladesh

mohammed.sayed.sameer@g.bracu.ac.bd

Abstract—Multi-class text classification is a fundamental problem in natural language processing with wide-ranging applications such as question categorization, document organization, and information retrieval. Unlike binary classification, multi-class classification requires models to distinguish between several semantically related categories, making the task more challenging. In this project, we conduct a comprehensive comparison between traditional machine learning approaches and neural network-based models for multi-class text classification. Two different word representation techniques are explored: Term Frequency–Inverse Document Frequency (TF-IDF) and Skip-gram embeddings generated using Word2Vec.

Logistic Regression is used as a representative machine learning baseline, while several neural architectures including Deep Neural Networks (DNN), Simple Recurrent Neural Networks (SimpleRNN), Gated Recurrent Units (GRU), Long Short-Term Memory (LSTM) networks, and their bidirectional variants are implemented. All models are evaluated using accuracy, macro-averaged F1-score, confusion matrices, and classification reports. Experimental results demonstrate that neural models using Skip-gram embeddings outperform traditional TF-IDF-based approaches, with Skip-gram + Bidirectional GRU achieving the best overall performance due to its stable gated recurrence and bidirectional context modeling.

Index Terms—Multi-Class Text Classification, TF-IDF, Skip-gram, Word2Vec, Neural Networks, LSTM, Natural Language Processing

I. INTRODUCTION

Multi-class text classification requires distinguishing among multiple semantically related categories, making it more challenging than binary classification. Traditional approaches rely on sparse representations such as TF-IDF, which are efficient but do not capture semantic similarity or word order. In contrast, distributed embeddings such as Skip-gram Word2Vec encode semantic structure and can be paired with neural sequence models to capture contextual dependencies. This work compares TF-IDF based models with Skip-gram embedding based neural architectures (SimpleRNN, GRU, LSTM, and bidirectional variants) to analyze how representation and architecture affect performance on a ten-class question–answer dataset.

II. DATASET AND EXPLORATORY DATA ANALYSIS

A. Dataset Description

The dataset provided by the course instructors consists of question–answer style text labeled into ten predefined topical categories. The data is pre-divided into training and testing splits, which were strictly preserved throughout all experiments to avoid data leakage. The training set contains 59,999 samples, while the test set contains 93,333 samples. Each instance includes a raw text field (*QA Text*) and a corresponding class label (*Class*), forming a ten-class classification task.

B. Exploratory Data Analysis

Class distribution analysis shows that the training dataset is nearly perfectly balanced, with approximately 6,000 samples per class. The test dataset exhibits mild variation across classes (approximately 9,290–9,419 samples per class). Although the imbalance is not severe, macro-averaged F1-score was used alongside accuracy to ensure balanced evaluation across all categories.

Text length analysis reveals substantial variability, with a right-skewed distribution indicating that most samples are relatively short while a minority contain long documents. Categories such as Politics & Government and Society & Culture tend to include longer texts, whereas Sports and Entertainment & Music are generally shorter. This motivated the use of padding and truncation when training neural sequence models.

Frequent token inspection highlights the presence of common stopwords and formatting artifacts such as “br” and “html,” indicating noise in the raw text. These findings justified the application of text cleaning and stopword removal during preprocessing.

C. EDA Summary

Overall, the dataset is large-scale, multi-class, and linguistically diverse. The near-balanced class distribution supports fair model training, while variability in text length and the presence of noisy tokens informed preprocessing choices, evaluation

metric selection, and sequence handling strategies for neural models.

III. TEXT PREPROCESSING

Two pipelines were evaluated: (P1) normalization + stop-word removal and (P2) P1 plus lemmatization. Results depended on representation: TF-IDF models performed best with lemmatization (P2), likely due to reduced vocabulary sparsity, while Skip-gram neural models performed best without lemmatization (P1), which preserved surface forms useful for embedding training. Accordingly, P2 was used for TF-IDF experiments and P1 for Skip-gram experiments.

IV. EXPERIMENTAL SETUP

To ensure reliable model evaluation and fair hyperparameter tuning, the training dataset was further divided into training and validation subsets. An 80:20 split was applied using stratified sampling based on class labels, ensuring that class proportions remained consistent across both subsets. As a result, 47,999 samples were used for training and 12,000 samples were reserved for validation.

Stratification was particularly important given the multi-class nature of the dataset, as it prevented class distribution skew during validation. The validation set was used exclusively for hyperparameter tuning and model selection, while the official test set was strictly preserved for final evaluation.

A. Evaluation Metrics

Model performance was evaluated using test accuracy and macro-averaged F1-score. Accuracy measures overall classification correctness, while macro F1-score ensures equal weighting across all classes regardless of class frequency. This is particularly important given the multi-class nature of the dataset.

V. TF-IDF WITH MACHINE LEARNING MODEL

A. TF-IDF Representation

Term Frequency–Inverse Document Frequency (TF-IDF) was employed as the primary word representation technique for traditional machine learning models. TF-IDF assigns higher importance to words that are frequent within a document but relatively rare across the corpus, making it effective for capturing discriminative terms [9].

Multiple TF-IDF configurations were explored by varying the maximum number of features and the n-gram range. Specifically, unigram and combined unigram–bigram representations were evaluated with feature sizes of 15,000 and 20,000. This experimentation allowed the model to capture both individual word importance and short contextual patterns.

B. Logistic Regression Classifier

Logistic Regression was selected as the baseline classifier due to its effectiveness on high-dimensional sparse data. The regularization parameter C was manually tuned using the validation set, and the optimal configuration ($C = 1.0$) was selected based on macro-averaged F1-score.

Using this configuration, the model achieved a test accuracy of approximately 68.1% and a macro F1-score of 0.68. These results demonstrate that TF-IDF combined with Logistic Regression provides a strong and computationally efficient baseline for multi-class text classification.

C. Bag-of-Words with Logistic Regression

A Bag-of-Words (BoW) based Logistic Regression model was implemented as a baseline for comparison. Although the model demonstrated stable training behavior, it consistently underperformed compared to TF-IDF based representations.

This result indicates that TF-IDF provides superior discriminative power by down-weighting frequent but less informative terms, and therefore BoW was not explored further in subsequent experiments.

D. Hyperparameter Tuning for TF-IDF + Logistic Regression

Hyperparameter tuning was performed using the validation set to identify the optimal TF-IDF configuration and regularization strength for the Logistic Regression classifier. Unigram and combined unigram–bigram representations were evaluated with different vocabulary sizes, and the regularization parameter $C \in \{0.5, 1.0\}$ was explored.

The best-performing configuration used 20,000 unigram TF-IDF features with $C = 1.0$, achieving the highest validation macro-averaged F1-score. This configuration was selected for final training and test evaluation.

E. Final Evaluation on Test Set

Using the optimal hyperparameters, the Logistic Regression model was retrained on the full training set and evaluated on the unseen test set. The model achieved an accuracy of approximately 68.1% and a macro F1-score of approximately 0.68. The confusion matrix and classification report indicate strong performance in categories such as Sports and Computers & Internet, while relatively lower performance was observed in semantically overlapping categories such as Education & Reference and Society & Culture.

These results demonstrate that TF-IDF combined with Logistic Regression provides a strong and computationally efficient baseline for multi-class text classification. These results highlight that while TF-IDF is effective for linear models, neural architectures benefit significantly more from dense embedding representations that preserve semantic and sequential information.

VI. TF-IDF WITH DEEP NEURAL NETWORK

A. Model Architecture

In addition to traditional machine learning, a feed-forward Deep Neural Network (DNN) was trained using TF-IDF feature vectors. Since neural networks require dense input representations, sparse TF-IDF vectors were converted to dense matrices prior to training. Class labels were encoded using integer encoding to support categorical loss functions.

The DNN architecture consists of two fully connected hidden layers with ReLU activation functions, followed by

dropout layers to mitigate overfitting. The output layer uses a softmax activation function to perform multi-class classification across ten categories. The model was trained using the Adam optimizer with sparse categorical cross-entropy loss [11].

B. Hyperparameter Tuning

Hyperparameter tuning for the TF-IDF based Deep Neural Network was performed using the validation set. Unigram TF-IDF features with a maximum vocabulary size of 5,000 were used, and sparse vectors were converted to dense matrices for neural network training.

Two configurations were evaluated by varying the number of hidden units, dropout rate, learning rate, and batch size. Early stopping with a patience of two epochs was applied based on validation loss to reduce overfitting.

The configuration with increased hidden units and a lower learning rate achieved the highest validation macro-averaged F1-score and was selected for final test evaluation.

C. Final Evaluation on Test Set

The optimized DNN model was evaluated on the test dataset using the same evaluation metrics as the machine learning baseline. The TF-IDF + DNN model achieved an accuracy of approximately 67.9% and a macro F1-score of approximately 0.67. While the neural model demonstrated competitive performance, its improvement over Logistic Regression was marginal.

This result suggests that although deep neural networks are powerful, TF-IDF representations may limit their ability to fully leverage contextual and sequential information, motivating the use of embedding-based representations in subsequent experiments.

VII. SKIP-GRAM EMBEDDINGS WITH RECURRENT NEURAL NETWORK MODELS

While TF-IDF captures word importance based on frequency statistics, it does not model semantic similarity or word order explicitly. To address this limitation, Skip-gram word embeddings were generated using Word2Vec [2]. These embeddings capture semantic relationships by learning vector representations where words appearing in similar contexts are placed closer in the embedding space. The learned embeddings were then used as input to sequential neural models, including SimpleRNN, GRU, LSTM, and their bidirectional variants.

A. Skip-gram Embedding Generation

Distributed word representations were learned using the Word2Vec framework. Both Continuous Bag-of-Words (CBOW) and Skip-gram models were initially explored for representation analysis; however, Skip-gram embeddings were selected for downstream classification due to their superior ability to capture semantic relationships for infrequent and context-sensitive words. Skip-gram embeddings were trained on the tokenized training corpus using the Word2Vec implementation provided by the `gensim` library [14], following the

model proposed by Mikolov et al. [2]. A vector dimension of 100 was chosen to balance representational power and computational cost. The embedding model was trained for 5 epochs with a context window size of 5 and a minimum word frequency threshold of 2.

Since the downstream neural models require embedding vectors aligned with the tokenizer vocabulary, an embedding matrix was constructed such that each token index maps to its learned Word2Vec vector. Tokens not present in the Word2Vec vocabulary were assigned zero vectors. The learned Skip-gram vectors were mapped to the tokenizer vocabulary to construct an embedding matrix for neural model initialization, providing a semantically meaningful starting point for supervised neural network training. CBOW embeddings were analyzed for representational comparison but were not used for downstream classification, as Skip-gram consistently yielded better validation performance for this dataset.

B. Sequence Preparation and Padding

Text samples were converted into sequences of word indices using a Keras tokenizer with a maximum vocabulary size of 20,000. To enable batch training with recurrent neural networks, sequences were padded or truncated to a fixed length.

The maximum sequence length was selected based on the 95th percentile of training sequence lengths, capped at 200 tokens to balance computational efficiency and information retention. Shorter sequences were padded using post-padding, while longer sequences were truncated using post-truncation. Class labels were converted into integer-encoded form using label encoding to enable the use of sparse categorical cross-entropy loss during neural network training. This encoding preserves class identity while avoiding the memory overhead of one-hot representations for a multi-class setting.

C. Recurrent Model Architecture

A unified neural architecture template was designed so that different recurrent layers could be swapped for fair comparison. Each model begins with an embedding layer initialized using the Skip-gram embedding matrix. This is followed by a recurrent layer (SimpleRNN, GRU, or LSTM) [3]–[5], optionally wrapped in a bidirectional layer for the bidirectional variants [6]. Dropout regularization was applied after the recurrent layer, followed by one or more dense layers with ReLU activation. The final classification layer uses a softmax activation function with ten output units corresponding to the ten classes.

All recurrent models were trained using the Adam optimizer [11] with a learning rate of 10^{-3} , sparse categorical cross-entropy loss, and accuracy as the primary training metric. Early stopping was applied with a patience of two epochs, restoring the best weights based on validation loss. Training was performed for up to 10 epochs with a batch size of 32.

D. Models Compared

The following Skip-gram embedding based neural network models were implemented and evaluated to analyze the impact

of different neural architectures on multi-class text classification performance. For each model, the best-performing hyperparameter setting identified using the validation set was selected for final comparison.

- Skip-gram + Deep Neural Network (DNN)
- Skip-gram + SimpleRNN
- Skip-gram + GRU
- Skip-gram + LSTM
- Skip-gram + Bidirectional SimpleRNN
- Skip-gram + Bidirectional GRU
- Skip-gram + Bidirectional LSTM

E. Training Behavior and Learning Dynamics

The training and validation loss and accuracy curves reveal substantial differences in learning stability across the evaluated neural architectures and hyperparameter settings, highlighting the sensitivity of recurrent models to optimization choices.

SimpleRNN-based models exhibit unstable training behavior, with validation accuracy remaining close to random guessing. These models fail to converge effectively due to their inability to preserve long-term dependencies and their susceptibility to vanishing gradient issues, even with early stopping and regularization.

Unidirectional LSTM models demonstrate a drastic change in learning behavior depending on hyperparameter choices. Under suboptimal settings, the model fails to converge and shows poor validation performance. However, after appropriate tuning, primarily through increased recurrent units and adjusted learning rates, the LSTM exhibits stable convergence and a significant improvement in both validation accuracy and macro F1-score. This indicates that LSTM performance is highly dependent on suitable model capacity and optimization settings.

GRU-based models display contrasting behavior across different hyperparameter selections. While some GRU configurations converge smoothly and achieve reasonable validation performance, others show training instability and a collapse in validation accuracy. This suggests that the GRU architecture is particularly sensitive to the interaction between learning rate and batch size, where suboptimal choices can lead to degraded generalization despite the presence of gating mechanisms.

Bidirectional architectures consistently demonstrate smoother convergence and improved stability compared to their unidirectional counterparts. Among all evaluated models, the Bidirectional GRU shows the most stable learning dynamics, with closely aligned training and validation curves and consistently high validation performance. This stability reflects the combined benefits of gated recurrence and bidirectional context modeling, which together enable effective learning from long and semantically diverse text sequences.

VIII. HYPERPARAMETER TUNING

A. Hyperparameter Tuning for Recurrent Models

All Skip-gram embedding based recurrent models were tuned using a two-phase unified strategy. In the first phase, each architecture was trained using a baseline set of commonly

used hyperparameters to establish reference performance and observe training stability. In the second phase, selected hyperparameters were adjusted based on validation performance and training behavior.

The hyperparameters explored include the number of recurrent units, learning rate, batch size, and dropout rate, while the embedding dimension was kept fixed to ensure comparability across models. Validation macro-averaged F1-score was used as the primary criterion for selecting the final configuration, as it provides balanced evaluation across all ten classes.

This two-phase tuning approach revealed substantial sensitivity to hyperparameter choices, particularly for gated recurrent architectures. While some models, such as LSTM, benefited significantly from tuning, others, such as GRU, exhibited degraded performance under certain configurations. These observations highlight the importance of systematic hyperparameter tuning for achieving stable and generalizable performance in multi-class text classification tasks.

IX. RESULTS AND MODEL COMPARISON

This section presents a comparative evaluation of all models based on their performance on the unseen test dataset. The comparison includes TF-IDF based machine learning models, TF-IDF based neural models, and Skip-gram embedding based neural architectures, each evaluated under their best-performing configurations.

For Skip-gram based neural models, results are reported for both the baseline and tuned settings to illustrate the impact of hyperparameter tuning on model performance and learning stability. To ensure fair assessment across all ten classes, both test accuracy and macro-averaged F1-score are reported. Macro F1-score is particularly important in this setting, as it assigns equal importance to each class regardless of class frequency and provides a balanced evaluation for multi-class classification tasks.

TABLE I
COMPARISON OF TF-IDF BASED MODELS ON TEST SET

Model	Test Accuracy	Macro F1-score
TF-IDF + Logistic Regression	0.681324	0.679100
TF-IDF + Deep Neural Network	0.670749	0.667146

Table I shows that TF-IDF combined with Logistic Regression outperforms the TF-IDF based Deep Neural Network in both accuracy and macro-averaged F1-score.

A. Overall Performance Comparison

The test performance of all Skip-gram embedding based neural models is compared using test accuracy and macro-averaged F1-score, as shown in Table II.

TABLE II
TEST PERFORMANCE OF SKIP-GRAM EMBEDDING BASED NEURAL
MODELS

Model	Test Accuracy	Macro F1-score
Skip-gram + BiGRU	0.694899	0.688082
Skip-gram + BiLSTM	0.693013	0.686500
Skip-gram + LSTM	0.679245	0.671823
Skip-gram + DNN	0.668788	0.662510
Skip-gram + BiSimpleRNN	0.652063	0.643432
Skip-gram + GRU	0.637159	0.631775
Skip-gram + SimpleRNN	0.108429	0.035217

Table II shows that gated recurrent architectures consistently outperform non-gated models, confirming the importance of gating mechanisms for mitigating vanishing gradient issues in long text sequences [4], [12].

Among all evaluated models, **Skip-gram + Bidirectional GRU** achieves the best overall performance, obtaining the highest test accuracy (0.694899) and macro-averaged F1-score (0.688082). This result highlights the effectiveness of combining gated recurrence with bidirectional context modeling for capturing long-range dependencies in question-answer style text.

Bidirectional variants consistently outperform their unidirectional counterparts, indicating that access to both past and future contextual information is particularly beneficial for this task. The Bidirectional LSTM follows closely, further reinforcing the advantages of bidirectional gated architectures.

The Skip-gram + Deep Neural Network provides a non-sequential neural baseline and achieves competitive performance (accuracy 0.668788, macro F1-score 0.662510). However, it remains inferior to GRU- and LSTM-based architectures, particularly their bidirectional variants, as the DNN does not explicitly model word order or temporal dependencies. Although Skip-gram embeddings encode semantic similarity, the pooling operation removes sequential structure, limiting contextual exploitation.

In contrast, SimpleRNN-based architectures perform poorly, with results close to random guessing for a 10-class classification task. This underscores the limitations of non-gated recurrence for modeling long-range dependencies in semantically rich text [4], [6].

B. Best and Worst Performing Models

Based on the sorted test results, the **best-performing model** is **Skip-gram + Bidirectional GRU**, which achieves the highest test accuracy (**0.694899**) and macro-averaged F1-score (**0.688082**). This result highlights the effectiveness of combining Skip-gram embeddings with gated recurrent units and bidirectional context modeling when trained with optimized hyperparameters, enabling the model to capture long-range contextual dependencies in text.

The **worst-performing model** is **Skip-gram + SimpleRNN**, which records a test accuracy of only **0.108429** and a macro F1-score of **0.035217**, values that are close to random guessing for a 10-class classification task. This outcome indicates that the basic recurrent structure lacks the

capacity to preserve long-term dependencies and struggles to model complex semantic patterns required for effective multi-class text classification.

C. Discussion of Observed Performance Trends

The observed performance differences are primarily driven by architectural design choices and sensitivity to hyperparameter tuning. Gated recurrent architectures such as GRU and LSTM consistently outperform the non-gated SimpleRNN, as gating mechanisms mitigate vanishing gradient issues and enable better modeling of long-range dependencies in text sequences.

Bidirectional models further improve performance across most architectures by incorporating contextual information from both preceding and succeeding tokens. This is particularly beneficial for question-answer style text, where informative cues may appear at any position within the sequence.

Overall, the strong performance of the Bidirectional GRU demonstrates that combining gated recurrence with bidirectional processing provides a stable and effective representation for multi-class text classification. In contrast, the variability observed in unidirectional recurrent models highlights the importance of careful hyperparameter selection for achieving reliable performance.

D. Class-wise Performance and Error Patterns

Class-wise analysis of the best-performing Skip-gram based model (Bidirectional GRU) shows higher precision and recall for categories with distinctive and domain-specific vocabulary, such as *Computers & Internet* and *Sports*. These classes contain clearer lexical signals and exhibit limited semantic overlap with others.

Conversely, broader and semantically overlapping categories such as *Education & Reference*, *Society & Culture*, and *Politics & Government* achieve comparatively lower performance. These categories often share similar discourse patterns and vocabulary, increasing ambiguity during classification.

Overall, most misclassifications arise from semantic overlap rather than lack of information, indicating that even advanced embedding-based recurrent models face challenges when class boundaries are conceptually diffuse. These observations indicate that model errors are primarily driven by semantic overlap between categories rather than insufficient representational capacity. This further motivates the use of a detailed per-class evaluation for the best-performing model to better understand remaining classification challenges.

X. DETAILED EVALUATION OF BEST MODEL

The Skip-gram embedding combined with the Bidirectional GRU achieved the highest performance among all evaluated models. To further analyze its behavior, a detailed per-class evaluation was conducted using the test set.

For all remaining models, accuracy, macro-averaged F1-score, confusion matrices, and classification reports were computed on the test set. Due to space constraints, only aggregate performance metrics are presented in the comparison section.

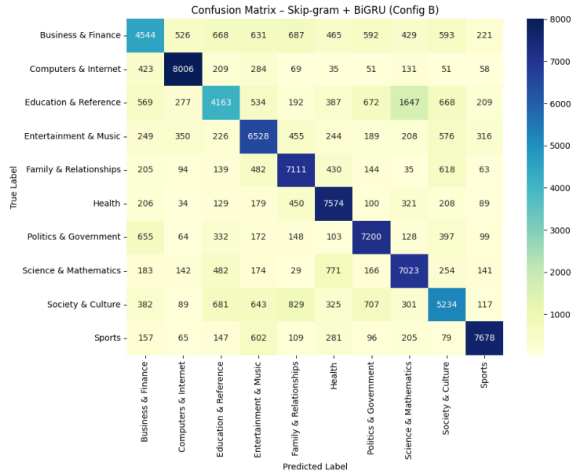


Fig. 1. Confusion matrix of the Skip-gram + Bidirectional GRU model on the test set

TABLE III
CLASSIFICATION REPORT OF THE SKIP-GRAM + BIDIRECTIONAL GRU MODEL

Class	Precision	Recall	F1-score	Support
Business & Finance	0.60	0.49	0.54	9356
Computers & Internet	0.83	0.86	0.84	9317
Education & Reference	0.58	0.45	0.50	9318
Entertainment & Music	0.64	0.70	0.67	9341
Family & Relationships	0.71	0.76	0.73	9321
Health	0.71	0.82	0.76	9290
Politics & Government	0.73	0.77	0.75	9298
Science & Mathematics	0.67	0.75	0.71	9365
Society & Culture	0.60	0.56	0.58	9308
Sports	0.85	0.82	0.83	9419
Macro Avg	0.69	0.70	0.69	—
Accuracy	0.70			

Overall, the experimental results consistently demonstrate the impact of representation choice, architectural design, and hyperparameter tuning on multi-class text classification performance. These findings provide a clear empirical foundation for the conclusions drawn in the following section.

XI. CONCLUSION

In this study, we conducted a comprehensive comparison of traditional machine learning models and neural network architectures for multi-class text classification using two word representation techniques: TF-IDF and Skip-gram embeddings. Experiments were performed on a large-scale ten-class question-answer dataset, and all models were evaluated using accuracy and macro-averaged F1-score on a held-out test set.

The results show that TF-IDF combined with Logistic Regression provides a strong and computationally efficient baseline, achieving competitive performance despite its simplicity and interpretability. However, TF-IDF representations are limited in their ability to capture word order and semantic

relationships, which restricts the effectiveness of deeper neural architectures when sparse features are used.

In contrast, Skip-gram embeddings combined with recurrent neural networks significantly improve classification performance by modeling both semantic similarity and sequential structure. Among all evaluated architectures, the **Skip-gram + Bidirectional GRU** achieves the best overall performance, demonstrating the effectiveness of gated recurrence combined with bidirectional context modeling under a well-tuned hyperparameter configuration.

The experimental findings further highlight that hyperparameter tuning plays a critical role in determining model stability and performance. While some architectures, such as LSTM, benefit substantially from tuning, others exhibit strong sensitivity to optimization settings, reinforcing the importance of systematic tuning rather than reliance on default parameters.

Overall, this work demonstrates that embedding-based neural architectures are better suited for capturing contextual dependencies in long and diverse text, while TF-IDF based models remain effective and interpretable baselines. Future work may explore transformer-based architectures, such as attention-based models or domain-adaptive pre-trained language models, to further improve multi-class text classification performance.

REFERENCES

- [1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [2] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2013.
- [3] J. L. Elman, "Finding structure in time," *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.
- [4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [5] K. Cho *et al.*, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, 2014, pp. 1724–1734.
- [6] M. Schuster and K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [7] F. Chollet *et al.*, *Keras*. Available: <https://keras.io>
- [8] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. OSDI*, 2016, pp. 265–283.
- [9] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [10] A. Y. Ng, "Feature selection, L1 vs. L2 regularization, and rotational invariance," in *Proc. 21st Int. Conf. Machine Learning (ICML)*, 2004, pp. 78–85.
- [11] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. OSDI*, 2016, pp. 265–283.
- [12] K. Cho *et al.*, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, 2014, pp. 1724–1734.
- [13] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2013.
- [14] R. Řehůřek and P. Sojka, "Software framework for topic modelling with large corpora," in *Proc. LREC Workshop on New Challenges for NLP Frameworks*, 2010.